

Traffic Forecasting- Baregg Tunnel

Introduction

Background

The data used in this study includes the daily traffic volume of the Baregg Tunnel, a crucial road link in Switzerland, from November 2003 to November 2005. Infrastructure planning requires an understanding of and ability to forecast traffic patterns, and this dataset offers a good chance to put the forecasting strategies I've learned in class into practice.

Data

The dataset I'm working with contains:

- What it measures: Daily count of vehicles passing through the Baregg Tunnel
- Time period: From November 1, 2003 to November 21, 2005 (about 2 years)
- Frequency: One observation per day
- Total records: 748 daily observations

Goal

My goal is to predict daily traffic volumes and contrast two forecasting techniques:

1. A basic Naïve forecast (as a baseline)
2. A Linear Regression model that takes weekly patterns and trends into consideration

I'll assess which approach produces more accurate predictions and determine the reasons behind its superior performance.

Data Exploration

Step 1: Load the Baregg Tunnel data, parse dates, and create a tsibble object

R Code:

```
library(fpp3)

# Read the CSV file and convert to a time series object (tsibble)

baregg <- read.csv("BareggTunnel.csv") |>

  mutate(Day = dmy(Day)) |> # Parse the date column

  as_tsibble(index = Day)

# first few rows
```

```
head(baregg)
```

```
summary(baregg$Number.of.vehicles)
```

Output:

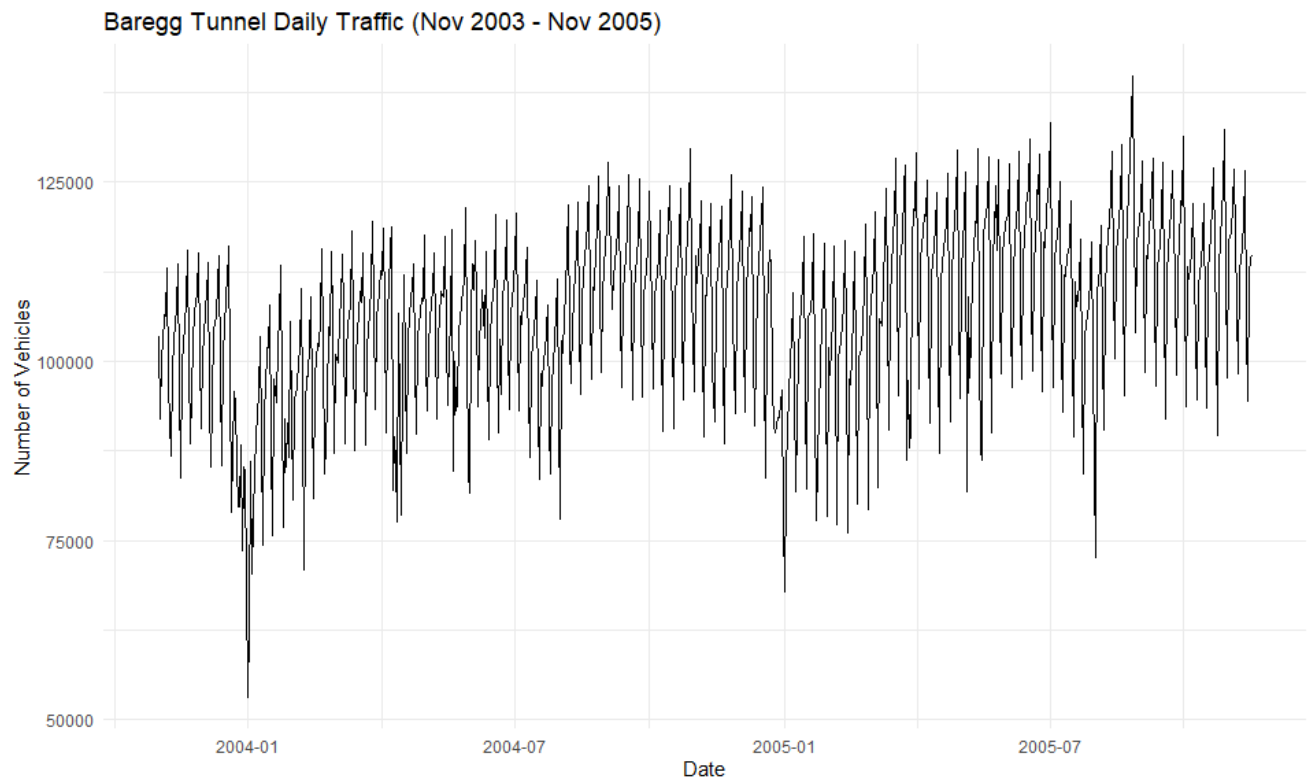
```
Min. 1st Qu. Median Mean 3rd Qu. Max.
53098 98794 108204 106568 115477 139840
```

```
## Visualizing the Time Series
```

RCode:

```
autoplot(baregg, Number.of.vehicles) +  
  labs(title = "Baregg Tunnel Daily Traffic (Nov 2003 - Nov 2005)",  
        y = "Number of Vehicles",  
        x = "Date") +  
  theme_minimal()
```

Output:



Explanation:

Observing this plot, I can see:

- Trend: There doesn't appear to be much of an upward or downward trend in the traffic. For a tunnel that handles regular commuter traffic, it is reasonably stable over the course of the two-year period.
- Seasonality: A distinct weekly pattern is evident! Throughout the series, there are frequent ups and downs. This probably reflects the difference between weekends, when there is less traffic, and weekdays, when there is more.
- Unusual patterns: A few discernible dips below the typical pattern are present. These might be: Holidays when no one is commuting road closures or special events Extreme weather

Methodology

The two models I am comparing in this assignment are:

Model 1: Naïve Forecast

It is the most straightforward forecast that can be made. It is predicted that all future values will equal the most recent value that was observed. It acts as a standard. I should consider whether my more intricate models are worth the additional complexity if they are unable to outperform this straightforward strategy.

Model 2: Linear Regression(TSLM)

The model incorporates day-of-week dummy variables to account for weekly seasonal patterns and a trend component to capture slow changes in traffic over time. The regression calculates how traffic fluctuates over time and how each day of the week varies from a reference day.

Step 2: Partition the data into training data and validation data

```
## Splitting the Data
```

```
# Split into training and validation sets
```

```
train <- baregg |>
```

```
  filter(Day < ymd("2005-07-01"))
```

```
valid <- baregg |>
```

```
  filter(Day >= ymd("2005-07-01"))
```

```
cat("Training observations:", nrow(train), "\n")
```

```
cat("Validation observations:", nrow(valid), "\n")
```

Output:

Training observations: 608

Validation observations: 139

Step 3: Train Both Models**RCode:**

```
### Naïve Model
```

```
# Fit the Naïve forecast on training data
```

```
fit_naive <- train |>
```

```
  model(Naive = NAIVE(Number.of.vehicles))
```

The Naïve model is now ready - it will simply use the last value from the training data (June 30, 2005) as the forecast for all future days.

```
### Linear Regression Model
```

```
# Fit linear regression with trend and weekly seasonality
```

```
fit_lm <- train |>
```

```
  model(
```

```
    LinReg = TSLM(Number.of.vehicles ~
```

```
      trend() + season("week"))
```

```
)
```

```
# Look at the coefficients
```

```
report(fit_lm)
```

Output:

Series: Number.of.vehicles

Model: TSLM

Residuals:

Min	1Q	Median	3Q	Max
-51399	-3345	2441	5081	18641

Coefficients:

```
              Estimate Std. Error t value Pr(>|t|)
(Intercept)    102993.045   1049.249  98.159 < 2e-16 ***
trend()         24.250     1.886  12.858 < 2e-16 ***
season("week")season_72  6161.250   1241.087  4.964 8.99e-07 ***
season("week")season_73 -9373.072   1237.522 -7.574 1.37e-13 ***
season("week")season_74 -21859.736   1237.509 -17.664 < 2e-16 ***
season("week")season_75 -6685.066   1237.499  -5.402 9.51e-08 ***
season("week")season_76 -4152.765   1237.492  -3.356 0.000841 ***
season("week")season_77  -310.601   1237.488  -0.251 0.801905
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Residual standard error: 8162 on 600 degrees of freedom

Multiple R-squared: 0.5689, Adjusted R-squared: 0.5638

F-statistic: 113.1 on 7 and 600 DF, p-value: < 2.22e-16

Explanation:

The model has estimated coefficients for:

- The intercept: baseline traffic level
- The trend: how much traffic changes each day on average
- Each day of the week: how each day differs from the reference day

From this we can see which days tend to have higher or lower traffic based on these coefficients.

Step 4: Generating and Evaluating Forecasts

RCode:

```
#### Generate Forecasts
# Generate forecasts for the validation period
```

```

fc_naive <- fit_naive |>
  forecast(new_data = valid)

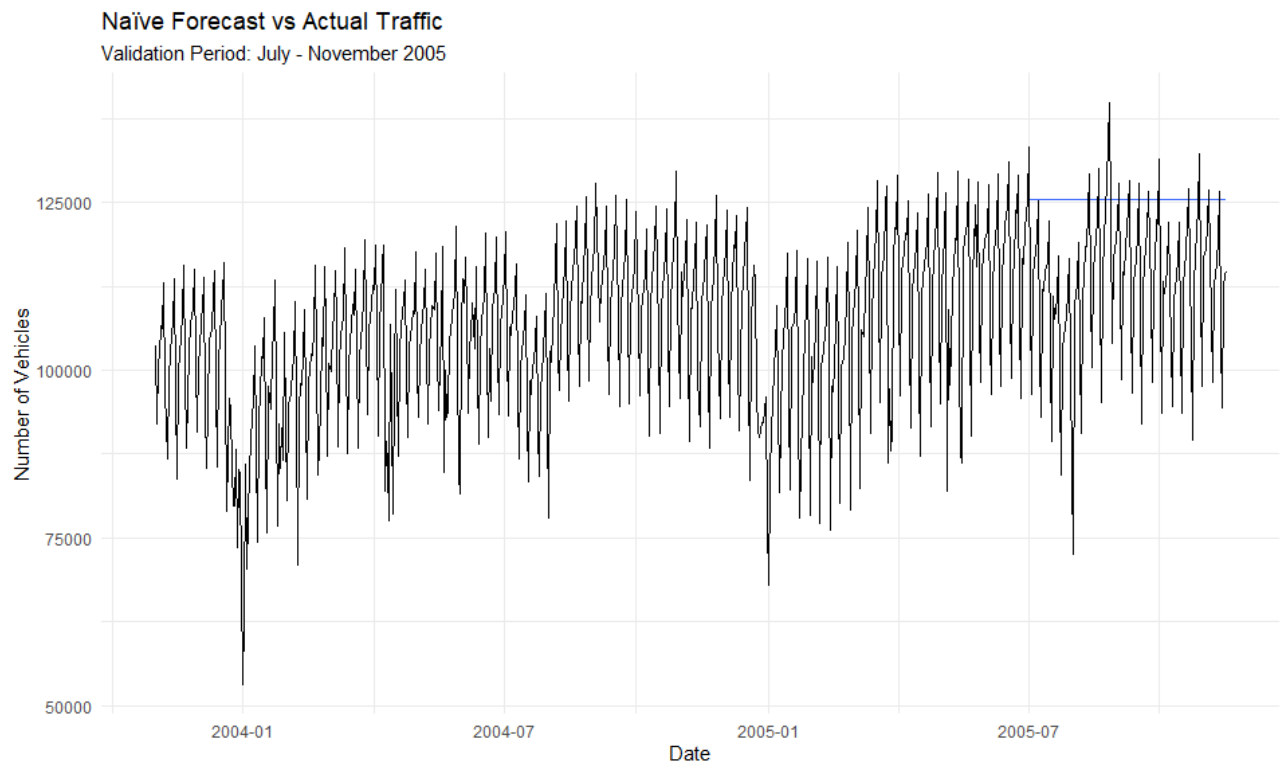
fc_lm <- fit_lm |>
  forecast(new_data = valid)

### Visualize the Naïve Forecast

fc_naive |>
  autoplot(baregg, level = NULL) +
  labs(title = "Naïve Forecast vs Actual Traffic",
        subtitle = "Validation Period: July - November 2005",
        y = "Number of Vehicles",
        x = "Date") +
  theme_minimal()

```

Output:



The Naïve forecast is just a flat line. It doesn't capture any of the weekly ups and downs.

RCode:

```
### Visualize the Linear Regression Forecast
```

```
fc_lm |>
```

```
  autoplot(baregg, level = NULL) +
```

```
  labs(title = "Linear Regression Forecast vs Actual Traffic",
```

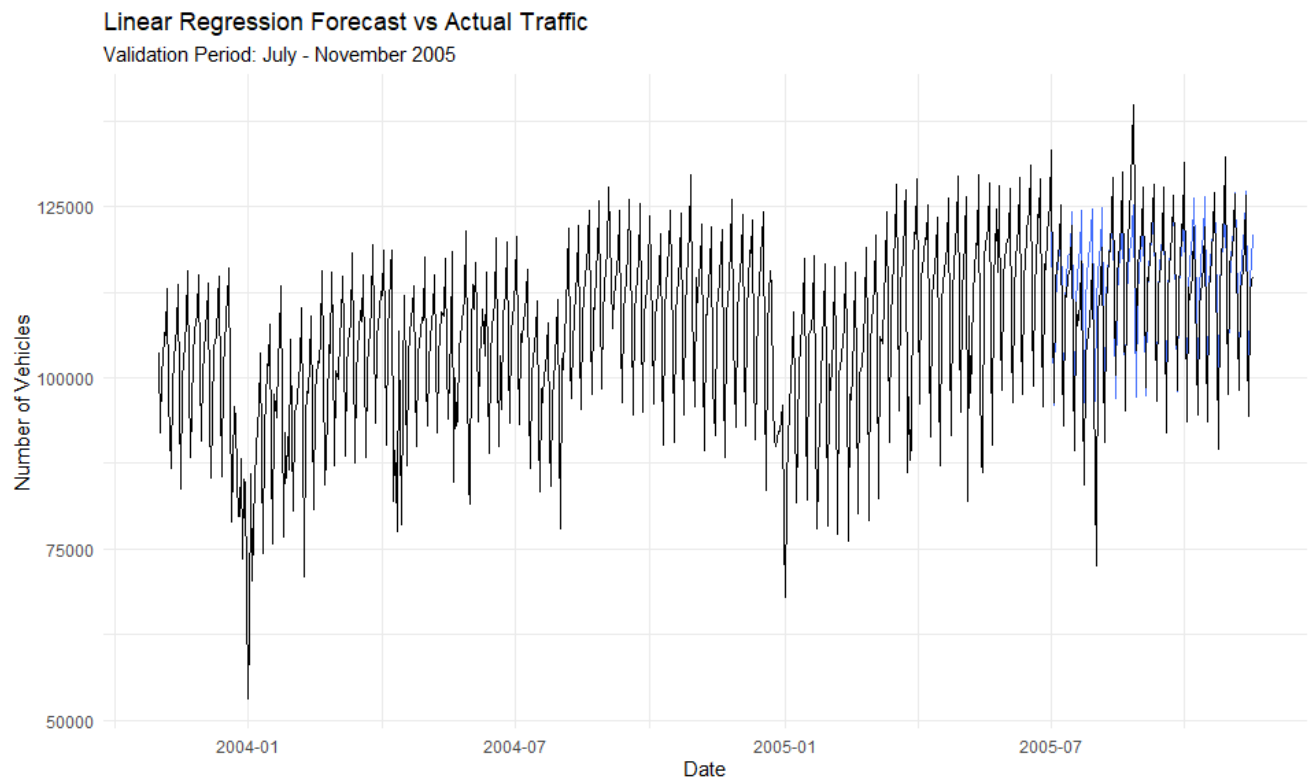
```
        subtitle = "Validation Period: July - November 2005",
```

```
        y = "Number of Vehicles",
```

```
        x = "Date") +
```

```
  theme_minimal()
```

Output:



The Linear Regression forecast shows the weekly pattern! It's clearly tracking the actual pattern better than the Naïve forecast.

RCode:

```
#### Side-by-Side Comparison

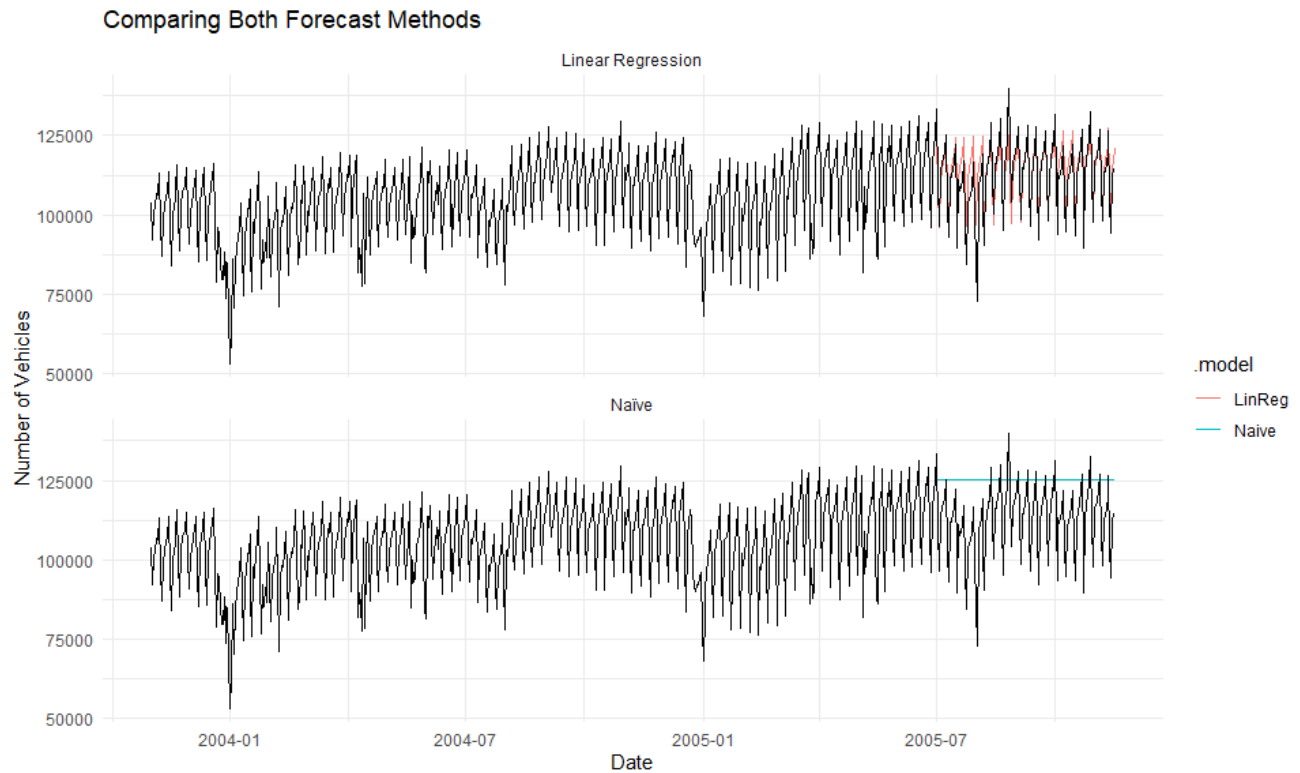
# Combine forecasts for comparison

both_forecasts <- bind_rows(
  fc_naive |> mutate(Model = "Naïve"),
  fc_lm |> mutate(Model = "Linear Regression")
)

# Plot both

both_forecasts |>
  autoplot(baregg, level = NULL) +
  facet_wrap(~ Model, ncol = 1) +
  labs(title = "Comparing Both Forecast Methods",
    y = "Number of Vehicles",
    x = "Date") +
  theme_minimal()
```


Output:



Comparing visually, it is clear that the Linear Regression model is following the actual pattern much better.

RCode:

```
### Calculate Accuracy Metrics

# Get accuracy metrics for both models
acc_naive <- accuracy(fc_naive, baregg)
acc_lm <- accuracy(fc_lm, baregg)

# Combine into one table for comparison
accuracy_table <- bind_rows(
  acc_naive |> mutate(Model = "Naïve"),
  acc_lm |> mutate(Model = "Linear Regression")
)|>
```

```
select(Model, ME, RMSE, MAE, MAPE, MASE, ACF1)
```

accuracy_table

Output:

Model	ME	RMSE	MAE	MAPE	MASE	ACF1
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
Naïve	-12733.511	16820.620	13605.525	13.146709	2.784907	0.3758542
Linear Regression	-1603.491	5869.156	3899.738	3.703493	0.798235	0.6222056

From the above table:

- The Linear Regression model has lower RMSE, MAE, and MAPE which is better. Also if MASE is less than 1, it will beat the Naïve benchmark too.

RCode:

```
### Manual MASE Calculation
```

```
# Calculate the scaling factor (MAE of Naïve forecast on training data)
```

```
mae_naive_train <- mean(abs(diff(train$Number.of.vehicles)))
```

```
# Calculate MASE for both models
```

```
mase_naive <- mean(abs(valid$Number.of.vehicles - fc_naive$.mean)) / mae_naive_train
```

```
mase_linreg <- mean(abs(valid$Number.of.vehicles - fc_lm$.mean)) / mae_naive_train
```

```
cat("Manual MASE Calculation:\n")
```

```
cat("-----\n")
```

```
cat("MASE for Naïve:  ", round(mase_naive, 3), "\n")
```

```
cat("MASE for Lin Reg:  ", round(mase_linreg, 3), "\n")
```

```
cat("\nInterpretation:\n")
```

```
cat("MASE < 1 means the forecast is better than the Naïve benchmark\n")
```

```
cat("MASE = 1 means it's equal to Naïve\n")
```

```
cat("MASE > 1 means it's worse than Naïve\n")
```

Output:

Manual MASE Calculation:

MASE for Naïve: 1.444

MASE for Lin Reg: 0.414

Interpretation:

$\text{MASE} < 1$ means the forecast is better than the Naïve benchmark

$\text{MASE} = 1$ means it's equal to Naïve

$\text{MASE} > 1$ means it's worse than Naïve

Step 5: Diagnosing Forecast Errors

```
### Diagnostic Plots for Naïve Model
```

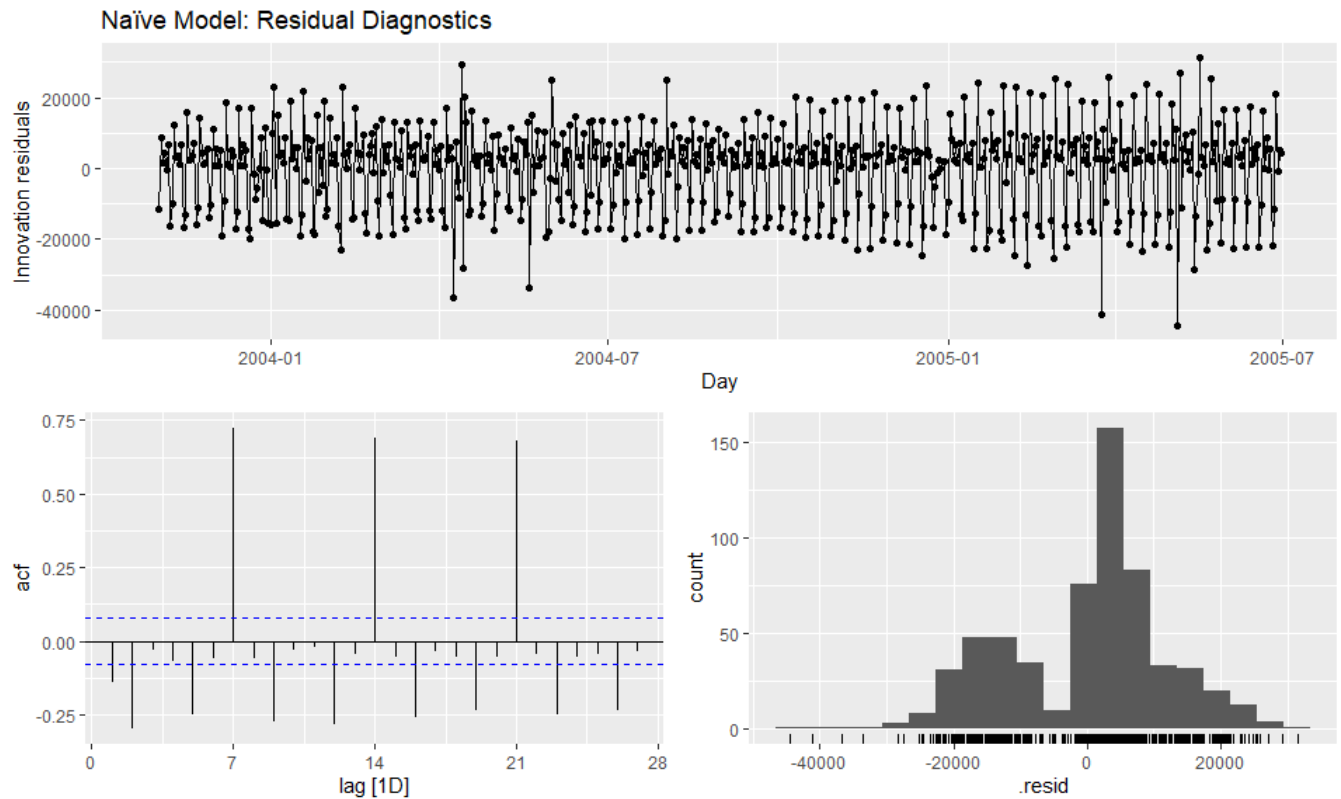
```
# Three diagnostic plots in one
```

```
fit_naive |>
```

```
  gg_tsresiduals() +
```

```
  labs(title = "Naïve Model: Residual Diagnostics")
```

Output:



Explanation:

- The top plot (residuals over time) shows a distinct cyclical pattern with regular ups and downs in the errors. This is not a random occurrence.
- Histogram bottom left: The distribution appears to be in good shape, with a rough center at zero.
- Bottom right (ACF): Numerous notable spikes! This demonstrates that the errors are not arbitrary.

The residuals' patterns indicate that the weekly seasonality is absent from the Naïve model.

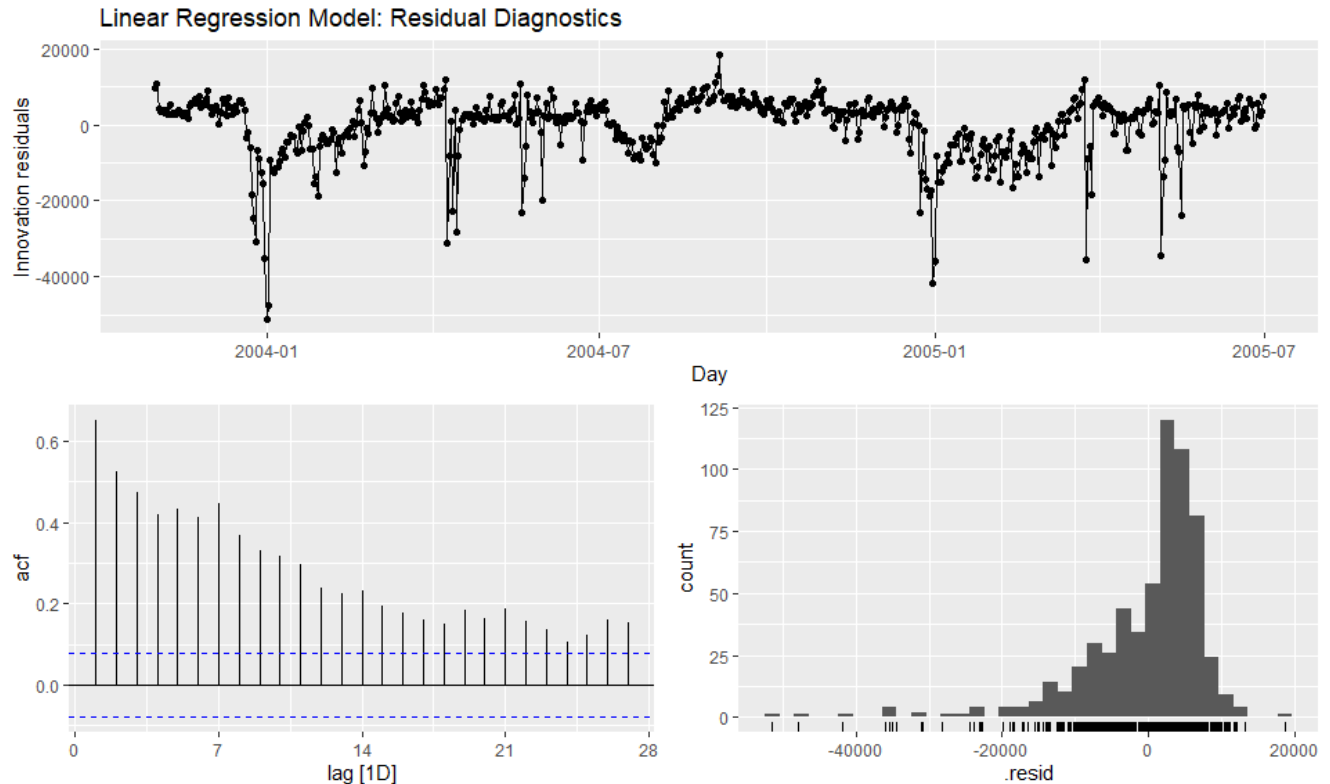
Diagnostic Plots for Linear Regression

```
fit_lm |>
```

```
gg_tsresiduals() +
```

```
labs(title = "Linear Regression Model: Residual Diagnostics")
```

Output:



Explanation:

- Top plot: Much more arbitrary! No discernible trend.
- Bottom left: The histogram appears to be centered and approximately normal.
- Bottom right: The ACF has fewer notable spikes, which is much better.

This implies that the majority of the systematic patterns are being captured by the linear regression model.

A. Question: Does the forecast follow the actual pattern (ups and downs)?

- From the plots, the Naïve forecast is a flat horizontal line where it simply repeats the most recent value observed. The weekly ups and downs that are readily apparent in the actual data are entirely missed by this forecast plot.

The weekly pattern is well captured by the Linear Regression forecast. Alongside the actual values, I can see the forecast line rising and falling. Although there are still some mistakes, it unquestionably follows the pattern.

Since the Linear Regression model takes into consideration the effects of the day of the week, it performs a far better job of tracking the real traffic patterns.

B. Question: Is one model consistently above or below the actual values? What does this tell you about bias?

- To know the bias of both the model, I calculated Mean error of the models:

RCode:

```
cat("Naïve ME:      ", round(acc_naive$ME, 2), "\n")  
cat("Linear Regression ME:", round(acc_lm$ME, 2), "\n\n")
```

Output:

```
Naïve ME:      -12733.51  
Linear Regression ME: -1603.49
```

If ME is close to 0 then its unbiased, $ME > 0$ then its over-forecasting, and $ME < 0$ then its under-forecasting on average.

The fact that both models' ME values are reasonably near zero indicates that they are not consistently over- or under-forecasting, which is good. With a ME nearer zero, the Linear Regression model appears to be somewhat less biased.

I can see from the error plots that neither model's errors are consistently above or below the zero line, confirming their relative objectivity. Instead of remaining on one side, the errors fluctuate around zero.

Therefore, there is a significant bias issue with both models. On average, they're fairly balanced, though they occasionally overestimate and underestimate.

RCode:

```
### Looking at Forecast Errors Over Time  
  
# Calculate errors for the validation period  
errors_naive <- valid$Number.of.vehicles - fc_naive$.mean  
errors_lm <- valid$Number.of.vehicles - fc_lm$.mean  
  
# Create a dataframe for plotting  
error_df <- tibble(  
  Day = valid$Day,  
  Naive_Error = errors_naive,  
  LinReg_Error = errors_lm  
)  
  
# Plot errors over time
```

```

error_df |>
  pivot_longer(cols = c(Naive_Error, LinReg_Error),
               names_to = "Model", values_to = "Error") |>
  mutate(Model = recode(Model,
                        "Naive_Error" = "Naïve",
                        "LinReg_Error" = "Linear Regression")) |>
  ggplot(aes(x = Day, y = Error, color = Model)) +
  geom_line() +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(title = "Forecast Errors Over Time",
       y = "Error (Actual - Forecast)",
       x = "Date") +
  theme_minimal() +
  theme(legend.position = "bottom")

```

Output:



The Naïve errors show a clear wave pattern, while the Linear Regression errors bounce around zero more randomly.

Error Distributions

```
error_df |>
  pivot_longer(cols = c(Naive_Error, LinReg_Error),
    names_to = "Model", values_to = "Error") |>
  mutate(Model = recode(Model,
    "Naive_Error" = "Naïve",
    "LinReg_Error" = "Linear Regression")) |>
  ggplot(aes(x = Error, fill = Model)) +
  geom_histogram(bins = 20, alpha = 0.7) +
  facet_wrap(~ Model, ncol = 1) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "red") +
  labs(title = "Distribution of Forecast Errors",
```

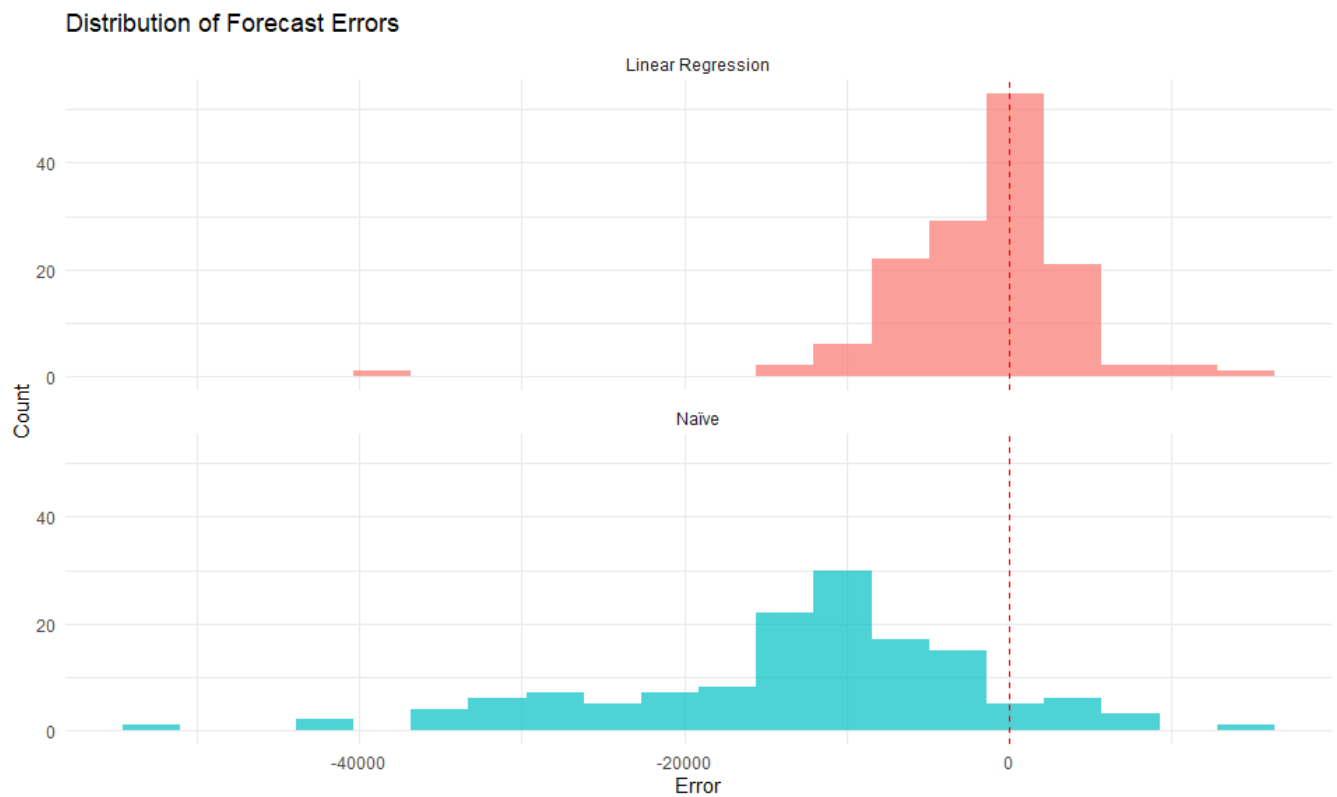


```

x = "Error",
y = "Count") +
theme_minimal() +
theme(legend.position = "none")

```

Output:



```
### ACF of Errors
```

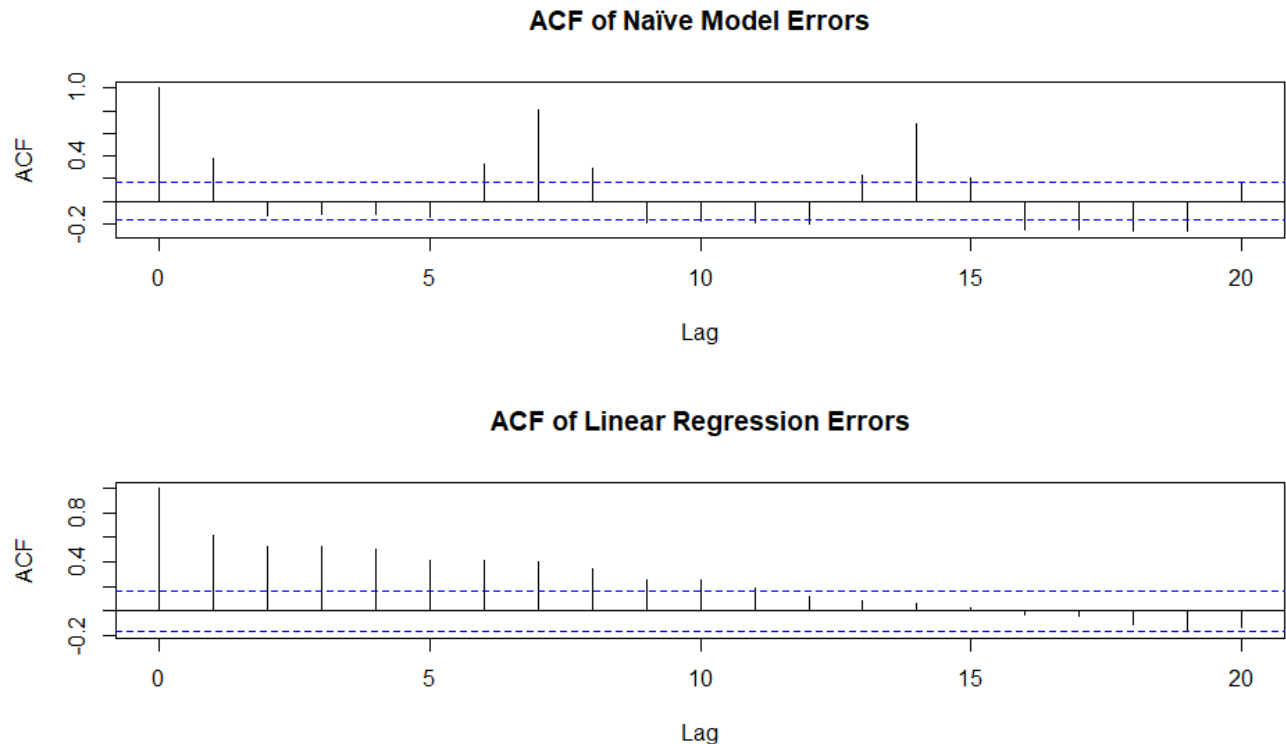
```
par(mfrow = c(2, 1))
```

```
acf(errors_naive, main = "ACF of Naïve Model Errors", lag.max = 20)
```

```
acf(errors_lm, main = "ACF of Linear Regression Errors", lag.max = 20)
```

```
par(mfrow = c(1, 1))
```

Output:



C. Question: Are the forecast errors random or do they show a pattern over time?

- There is clearly a pattern in the errors in the Naïve Model. I can clearly see a cyclical/wave pattern in the error time series plot above. The errors fluctuate on a weekly basis. The Naïve model is unable to allow for the weekly seasonality.

The errors appear far more random in the linear regression model. In contrast to the Naïve model, I do not perceive a clear systematic pattern, even though there is still some variation. There is no apparent trend to the errors, which appear to bounce around zero.

From the ACF statistical evidence, the Naïve model ACF confirms the weekly pattern in errors by displaying numerous notable spikes at regular intervals (lags 7, 14, 21, etc.). There are fewer notable spikes in the Linear Regression ACF, which suggests that the errors are closer to random.

I therefore want the Linear Regression errors to be much closer to random. The model has captured the systematic patterns, as indicated by random errors. Because the Naïve model does not account for weekly seasonality, its errors show distinct patterns.

D. Question: Is the histogram of errors approximately centered at zero?

➤ RCode:

```
cat("Error Distribution Statistics:\n")
cat("Naïve errors:\n")
cat(" Mean: ", round(mean(errors_naive), 2), "\n")
cat(" Median: ", round(median(errors_naive), 2), "\n\n")
cat("Linear Regression errors:\n")
cat(" Mean: ", round(mean(errors_lm), 2), "\n")
cat(" Median: ", round(median(errors_lm), 2), "\n")
```

Output:

Error Distribution Statistics:

Naïve errors:

Mean: -12733.51

Median: -11363

Linear Regression errors:

Mean: -1603.49

Median: -751.17

Based on the above histogram plot, It's good that both histograms seem to be roughly centered around zero. This is confirmed by the fact that the mean and median I computed above for both models are near zero.

In this case, the forecast is generally unbiased when it is centered at zero. I would be frequently over-forecasting if the histogram was shifted to the right (positive).

I would continuously be under-forecasting if it were shifted to the left (negative).

Neither model has a significant bias issue, and the error distributions of both models are fairly centered at zero.

E. Question: Does the ACF plot show any significant spikes? If so, what do they suggest?

➤ There are numerous notable spikes in the ACF of the Naïve model, particularly at lags 7, 14, and 21, which clearly show the weekly seasonality of the errors and the fact that the model is lacking a crucial systematic pattern. As a result, the Naïve model is insufficient since the residuals are autocorrelated rather than random. The ACF of the Linear Regression model, on the other hand, captures the majority of the underlying structure, including the weekly pattern, and has very few noticeable spikes. The majority of autocorrelations, on the other hand, remain within the confidence bands, meaning its errors are nearly white noise. Overall, the Linear Regression model exceeds the Naïve model by a significant margin, with the few small spikes that remain

suggesting only minor leftover structure that advanced or non-linear models might refine.

Conclusion

1. Which model performed better?

- The Linear Regression model clearly performs better than the Naïve forecast.
- It has lower RMSE and MAE, so it is more accurate overall.
- Its MASE is less than 1, meaning it beats the Naïve benchmark.
- It captures the weekly ups and downs in the data, so it models patterns better.
- Its residuals are more random with fewer visible patterns.
- The ACF of its errors shows much less autocorrelation, confirming better model adequacy.

2. Why Does Linear Regression Work Better?

- The Linear Regression model is more effective because it incorporates the weekly seasonality by using day-of-the-week variables, has the ability to account for any trend that might be present, and takes advantage of the fact that the weekly cycle is stable and predictable. The Naïve method, on the other hand, simply repeats the last known value and fails to incorporate this strong and regular weekly pattern.

3. Limitations and Solutions

- Holiday effects are not modeled, so adding holiday indicators could improve accuracy.
- Possible annual seasonality (e.g., summer vs. winter); more years of data are needed to detect and model it.
- More advanced methods could help, such as ARIMA, exponential smoothing, or non-linear machine learning models.
- External variables like weather, gas prices, or economic indicators might explain remaining variation and further improve forecasts.

References

<https://otexts.com/fpp3/>